

Dual-Modal Deep Learning and Stacking Ensemble for Zero-Day Phishing Detection

Mohammed Sulaiman Eksambi
CSE - Cybersecurity
Dayananda Sagar University
Harohalli, India
sulaimaneksambi@gmail.com

Numan Nisar Ahmed
CSE - Cybersecurity
Dayananda Sagar University
Harohalli, India
nisarnuman7@gmail.com

Dushanth Purushotham D
CSE - Cybersecurity
Dayananda Sagar University
Harohalli, India
dushanthpurushotham@gmail.com

Abstract—Traditional phishing detection mechanisms depend heavily on static blacklists, URL heuristics, and superficial visual matching, which are difficult to catch against modern, high-fidelity phishing kits that employ dynamic Single Page Application (SPA) frameworks. This paper presents a novel Deep Learning Stacking Ensemble that combines Convolutional Neural Networks (CNNs) and Transformer models (CodeBERT) for semantic and structural feature extraction, feeding into traditional machine learning models (XGBoost) to achieve maximum generalization on low-sample datasets. Furthermore, we also introduce autonomous, high-purity dataset generation pipeline capable of penetrating SPA frameworks and anti-bot systems to extract true credential-harvesting interfaces. Experimental results on a highly curated dataset yield a 94.12% accuracy and a 98.13% recall rate, demonstrating robust real-world applicability and high sensitivity to zero-day threats.

Index Terms—Phishing Detection, Deep Learning, Stacking Ensemble, CodeBERT, Convolutional Neural Networks, Cybersecurity.

I. INTRODUCTION

Phishing remains one of the most prominent vectors for cyberattacks, leading to massive credential theft and financial fraud. As enterprise web development has shifted towards complex, JavaScript-heavy frameworks (e.g., React, Vue), cybercriminals have adapted by deploying high-fidelity phishing kits that mirror these complex structures and actively evade automated scanners [5]. Standard image-recognition models often fail to differentiate between a visually identical phishing page and a benign enterprise login portal due to the noise of modern web design, such as inline SVGs and dynamic CSS.

To counter these sophisticated evasion tactics, this research proposes a structural and semantic approach to phishing detection. Rather than evaluating only the visual render or superficial heuristics, our system analyzes the underlying skeletal Document Object Model (DOM) routing and codebase to identify the true malicious intent of the page.

II. RELATED WORK

The detection of phishing websites has evolved significantly, but still existing methodologies struggle to keep pace with modern evasion techniques.

A. Visual and Heuristic-Based Detection

Early machine learning approaches relied heavily on manual feature extraction, analyzing URLs for anomalous character distributions or evaluating superficial webpage heuristics, such as the volume of `<script>` tags. More recent attempts have applied Convolutional Neural Networks (CNNs) to webpage screenshots to visually detect brand spoofing. However, models that evaluate only URLs or screenshots fail against high-fidelity visual clones, as a visually perfect clone can easily deceive a CNN despite its underlying malicious DOM routing.

B. Structural and DOM-Based Analysis

Analyzing the HTML structure or DOM to detect phishing is an established concept. Previous research has attempted to model the DOM tree using standard Natural Language Processing (NLP) or Graph Neural Networks (GNNs) to identify structural anomalies. While theoretically, the application of end-to-end deep learning models like GNNs on raw web structures introduces massive memory overheads, often converting complex HTML DOM trees into convoluted graph structures that create various computational bottlenecks [4].

C. The Gap in Existing Research

Current state-of-the-art (SOTA) models suffer from two critical limitations: structural blindness and dataset impurity. Most academic datasets contain massive amounts of noisy and defective data, including 404 error pages, CAPTCHA blockade pages, and empty JSON responses. Training models on these impure datasets leads to artificial accuracy metrics that collapse in real-world environments. Furthermore, end-to-end deep learning models demonstrate a massive tendency to overfit on highly curated, low-sample datasets, memorizing the training set rather than learning generalizable features. Our approach covers this gap by merging the deep feature extraction from the classification phase and implement strict dataset sanitation.

III. PROBLEM STATEMENT & LIMITATIONS OF CURRENT MODELS

Current state-of-the-art phishing detection models suffer from two critical limitations that limit and reduce their performance in live enterprise environments:

A. Dataset Impurity

Most academic datasets contain massive amounts of noisy or defective data, including 404 error pages, CAPTCHA blockades, and empty JSON responses. Training models on these datasets leads to artificial accuracy metrics that collapse in real-world scenarios.

B. Structural Blindness

Models that evaluate only URLs or screenshots fail against high-fidelity visual clones. A visually perfect spoof of a platform like Facebook can easily deceive a Convolutional Neural Network (CNN), but the underlying Document Object Model (DOM) routing—specifically where the harvested credentials are sent—will inevitably reveal anomalies. Our objective is to engineer a system that maintains strict dataset purity and evaluates the multi-dimensional structure of a webpage to achieve superior zero-day phishing detection on limited data samples.

IV. AUTONOMOUS DATASET GENERATION & SANITATION

To overcome the limitations of noisy datasets, we developed an autonomous, memory-managed data ingestion pipeline designed specifically to defeat modern web defenses.

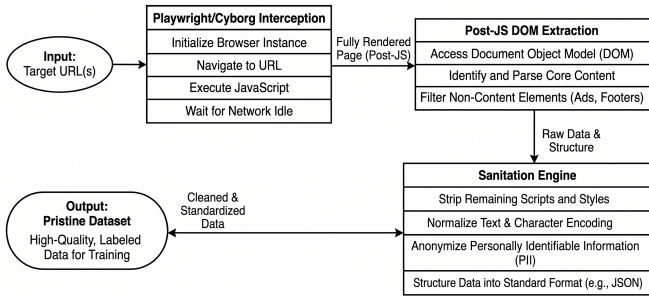


Fig. 1. Autonomous Dataset Generation and Sanitation Flowchart.

A. Penetrating SPAs & Anti-Bot Systems

Modern enterprise websites employ rigorous anti-bot systems like Cloudflare, which proactively block standard scraping libraries. As illustrated in Fig. 1, our pipeline utilizes Playwright injected with aggressive stealth configurations:

- **Engine Spoofing:** Utilizing the Firefox engine rather than Chromium to evade Chromium-specific automation fingerprinting.
- **Environmental Masking:** Dynamically aligning browser locale and timezone_id with the host IP geolocation to prevent Cloudflare challenge triggers.
- **Header Injection:** Enforcing strict, realistic HTTP headers (e.g., Sec-Fetch-Mode, Accept-Language) typical of standard human browsing behavior.

B. The “Cyborg” Manual Scraper

To capture highly complex enterprise logins that require physical human navigation (e.g., multi-step SSO, Microsoft login), we engineered a “Cyborg” manual scraper. It opens the browser with anti-bot configurations and injects a floating DOM button. A researcher navigates to the target form, bypasses any physical security present there, and triggers the button. The script intercepts the event and extracts the fully rendered, post-JavaScript DOM.

C. The Sanitation Engine

Dataset purity is strictly enforced via a multi-processing validation script. It structurally validates every HTML file, actively deleting files that are too small, missing a standard `<body>` tag, or lacking credential `<input>` fields. It utilizes MD5 hashing to detect and delete exact template duplicates, ensuring the neural network trains only on high quality and unique layouts.

V. ARCHITECTURAL EVOLUTION & KEY LEARNINGS

During the development of this model, a significant architectural pivot was required to address critical bottlenecks discovered during dataset auditing and initial training phases as the dataset the model was trained on was very small.

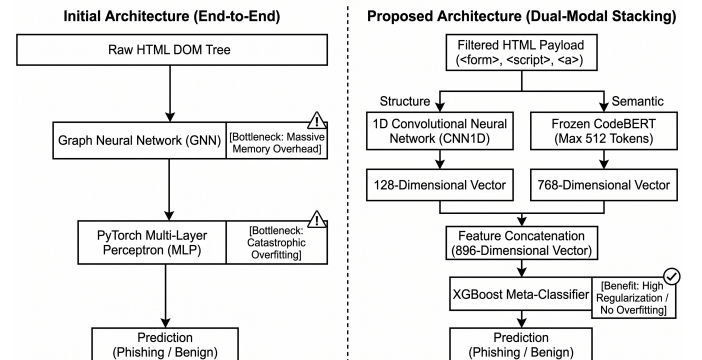


Fig. 2. Comparison between the high memory GNN pipeline (left) against the proposed decoupled Stacking Ensemble (right).

A. The Previous Architecture & Overfitting

The initial architecture used a end-to-end differentiable PyTorch Multi-Layer Perceptron (MLP) combined with a Graph Neural Network (GNN) to parse the raw DOM tree. However, end-to-end deep learning models are incredibly data-hungry. On our highly curated, low-sample dataset, the MLP demonstrated a massive tendency to overfit, memorizing the training set rather than learning generalizable features. Furthermore, the GNN introduced massive memory overheads by converting massive HTML DOM trees into complex graph structures [4].

B. The DOM Density Fallacy

A manual random audit of the dataset revealed a dangerous pitfall: legitimate benign websites are often massive, script-heavy applications, while many phishing sites are lightweight HTML clones. Initially, there was a concern that an end-to-end model would lazily classify based purely on file size and code density. However, the dataset audit revealed highly sophisticated phishing samples that perfectly mirror the complexity of real sites. Consequently, the model cannot rely on superficial metrics like DOM size; it must possess deep structural and semantic understanding, necessitating a pivot to a dual-modal architecture.

VI. PROPOSED SYSTEM ARCHITECTURE: DUAL-MODAL STACKING ENSEMBLE

To solve the overfitting problem while maintaining deep structural understanding, we abandoned the end-to-end architecture in favor of a Stacking Ensemble. The raw HTML payload is processed through two parallel neural networks and a routing heuristic engine to extract a dense 897-dimensional mathematical representation.

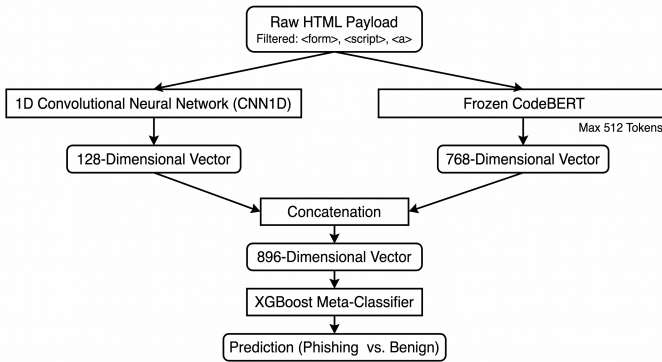


Fig. 3. System Architecture: Raw HTML is split into two deep learning paths (CNN1D and CodeBERT) alongside a deterministic routing heuristic. The resulting 128-dimensional, 768-dimensional, and 1-dimensional vectors are concatenated into an 897-dimensional vector for final classification via XGBoost.

A. Sequential Structure Extraction (CNN1D)

The first modality extracts local, sequential features from the raw HTML structure. By treating the code as a 1D sequence, the Convolutional Neural Network (CNN) learns to identify the spatial arrangements of tags and attributes that are highly characteristic of automated phishing kits [3]. This path produces a 128-dimensional vector.

B. Semantic Logic Extraction (Frozen CodeBERT)

The second modality utilizes a pre-trained Transformer model, CodeBERT [1], kept in a completely frozen state to prevent parameter overfitting. It analyzes the contextual meaning of variable names, functions, and text, identifying malicious logic disguised within the script. This produces a 768-dimensional vector.

Solving the 512-Token Limitation Bottleneck: CodeBERT possesses a strict 512-token input limit, whereas modern SPAs often exceed 10,000 tokens. We engineered a two-step “Distillation and Truncation” pipeline to bypass this constraint:

- 1) *Targeted Semantic Extraction:* Our parsing engine drops visual noise (e.g., `<div>`, `<img>`) and extracts exclusively `<form>` (prioritizing action attributes and routing destinations), `<script>`, and `<a>` elements, compressing the webpage into a pure logical structure.
- 2) *Priority Hard Truncation:* We enforce strict truncation (`max_length=512`). Because core architectural logic (form actions, initial states) almost exclusively occurs at the top of the DOM tree, CodeBERT successfully reads the “brain” of the operation while safely discarding trailing, irrelevant scripts.

C. Credential Routing Extraction (Heuristic)

To defend against perfect 1:1 visual clones that perfectly mirror enterprise mathematical structures, a third deterministic modality is introduced. This engine scans all `<form>` actions to detect malicious credential routing behaviors (e.g., raw IP addresses, free-hosting tunnels, or obfuscated PHP endpoints). This produces a critical 1-dimensional binary flag indicating if the underlying credential drop-zone is malicious.

D. Meta-Classification Fusion (XGBoost)

The concatenated 897-dimensional vectors are fed into a highly regularized XGBoost Classifier [2]. By separating the deep feature extraction from the classification phase, we leverage the complex understanding of deep learning while utilizing the strict mathematical boundaries of traditional Machine Learning, guaranteeing real-world generalization on small datasets.

VII. EXPERIMENTAL SETUP

A. Curated Dataset Parameters

To source the raw malicious URLs, a custom scraper was engineered to execute a manual, daily aggregation from the OpenPhish feed (<https://openphish.com/feed.txt>). This feed provided a constant stream of newly detected, zero-day phishing endpoints, which were subsequently fed into our autonomous ingestion pipeline.

Following heavy pruning to remove low-quality or dead samples, the final dataset resulted in 541 high-fidelity Benign pages. To maintain consistent experimental baselines, the Phishing dataset was algorithmically sorted and strictly capped to the first 700 valid samples. This configuration provides sufficient variance without requiring aggressive mathematical undersampling.

B. Training Metrics & Split

The dataset was strictly split into a 70/15/15 configuration:

- **70% Training Set:** Used for backpropagation on the CNN and XGBoost tree construction.

- **15% Validation Set:** Evaluated dynamically to halt epochs if validation loss increased, preventing memorization.
- **15% Test Set:** Strictly held back to generate final metrics.

VIII. RESULTS & EVALUATION

The model was evaluated against the unseen 15% Test Set. To prove the efficacy of the Stacking architecture, we simultaneously evaluated a Machine Learning Baseline (Random Forest relying on manual heuristics) on the exact same dataset split.

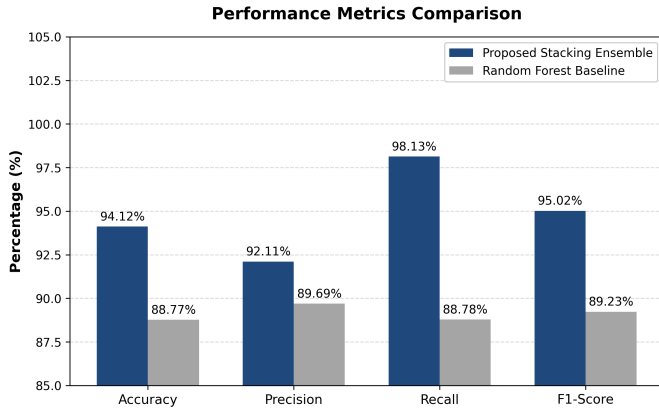


Fig. 4. Performance comparison across key metrics between the Random Forest baseline and the proposed Stacking Ensemble.

A. Comparative Analysis

As illustrated in Fig. 4, the experimental results demonstrate the superiority of the Stacking Ensemble. The Baseline model struggled to maintain a harmonic balance, achieving an accuracy of 88.77% and a recall of 88.78% while missing 11 actual phishing attacks, and generating 10 False Positives. In an enterprise environment, missing active threats while frequently blocking legitimate portals severely degrades trust. Conversely, the deep learning model’s semantic understanding of routing logic led to a superior F1-Score (95.02%) and an exceptional recall (98.13%). By missing only 2 complex threats out of the entire test set, the ensemble proves its highly robust capability to generalize and catch zero-day attacks.

Metric	Count	Description
True Negatives	71	Safe sites correctly identified.
False Positives	9	Safe sites incorrectly blocked (Minimal disruption).
False Negatives	2	Malicious sites that evaded detection.
True Positives	105	Malicious sites correctly blocked.

Metric	Count	Description
True Negatives	79	Safe sites correctly identified.
False Positives	10	Safe sites incorrectly blocked (Moderate disruption).
False Negatives	11	Malicious sites that evaded detection.
True Positives	87	Malicious sites successfully blocked.

IX. CONCLUSION & FUTURE WORK

The proposed Dual-Modal Stacking architecture demonstrates that extracting deep skeletal and semantic features using neural networks, and classifying them with traditional Machine Learning, creates an exceptionally robust defense against zero-day attacks. By strictly enforcing dataset purity through our automated sanitation pipeline, the model avoids the artificial inflation of accuracy caused by learning noisy pages. The fusion of frozen Transformers, CNNs, and XGBoost presents a highly optimized methodology for achieving maximum generalization on low-sample datasets in the fight against modern credential harvesting operations.

However, due to computational resource constraints during this research phase, the model was trained on a relatively small dataset. Future work will focus on scaling the infrastructure to ingest and train on a massively expanded dataset, which will further improve generalization capabilities and edge-case detection. Additionally, we aim to extend the semantic parsing engine to conduct comprehensive, multi-layered scans of the entire website architecture—analyzing all linked external source codes, deeply nested dependencies, and dynamic asset structures to definitively ascertain a website’s intent.

DATA AND CODE AVAILABILITY

To support the principles of reproducible research and open science, all artifacts associated with this study have been made publicly available.

- **Dataset:** The sanitized HTML dataset, containing both the benign and zero-day phishing samples, is hosted on Kaggle at: <https://www.kaggle.com/datasets/sulaimaneksambi/zero-day-phishing-post-rendered-html-dataset/data>
- **Pre-trained Model:** The extracted model weights and the frozen CodeBERT inference pipeline are available via Hugging Face at: <https://huggingface.co/XMB480/phishing-detector>
- **Source Code:** The complete source code, including the Playwright/Cyborg ingestion pipeline, sanitation engine, and Stacking Ensemble training scripts can be accessed on GitHub at: <https://github.com/sulaimanxmb/ZeroDay-Phishing-Ensemble>

REFERENCES

- Z. Feng et al., “CodeBERT: A Pre-Trained Model for Programming and Natural Languages,” *Findings of the Association for Computational Linguistics: EMNLP 2020*, pp. 1536–1547, 2020.
- T. Chen and C. Guestrin, “XGBoost: A Scalable Tree Boosting System,” in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pp. 785–794, 2016.

- [3] Y. Wang et al., “Phishing Website Detection Based on Deep Convolutional Neural Network and Random Forest Ensemble Learning,” *Sensors*, vol. 21, no. 24, p. 8281, 2021.
- [4] J. Zhang et al., “Phishing the Phishers with SpecularNet: Hierarchical Graph Autoencoding for Reference-Free Web Phishing Detection,” *arXiv preprint arXiv:2603.01874*, 2026.
- [5] A. Oest et al., “Sunrise to Sunset: Exploring the Lifespan and Predictability of Phishing URLs,” in *Proceedings of the Internet Measurement Conference*, pp. 134–147, 2020.